The Scalable Contract

Resolving beast3 issue #20: a binding three-method contract on `Scalable`

Alexei Drummond

5 May 2026

The blocker

Joelle Barido-Sottani has been blocked porting custom-tree packages to beast3 since 26 March.

Issue CompEvol/beast3#20, opened by Remco in October 2025.

Two distinct problems she ran into:

1. The old `Scalable` interface returns degrees of freedom (`int`), which encodes a specific Hastings-ratio shape (`dof` $\times$ `log(s)`). For her custom trees the HR isn't of that shape.
2. The new spec `UpDownOperator` bypasses `Tree.scale()` entirely and runs its own recursion. Even if she fixed her tree's `scale()`, UpDown wouldn't call it.

The old contract

```java
public interface Scalable {
    int scale(double s); // returns dof
    void scaleOne(int i, double s);
    default double scaleAll(double s) {
        return scale(s) * Math.log(s);
    }
}
```

What's wrong:

- `int dof` is a stand-in for one specific HR formula. Operators reconstruct `dof` $\times$ `log(s)` as the move's Jacobian contribution.
- Anything that scales differently (interval scaling, custom trees, IntervalScaleOperator) has to bypass the interface and compute HR inline.
- The interface gives no way to ask "what does this Scalable use as its dilation axis?"

What does `scale` actually return?

The Metropolis-Hastings acceptance ratio has three parts:

1. **Target ratio**: $\pi(x')/\pi(x)$.
2. **Proposal ratio** (the Hastings ratio proper): $q(x \mid x')/q(x' \mid x)$.
3. **Jacobian determinant**: $|\det(\partial x'/\partial x)|$ from the change of variables.

`Scalable.scale(s)` can only sensibly return the **Jacobian** of the deterministic part of the move.

The proposal ratio depends on how the operator drew `s` (uniform-on-log, Bactrian, Gaussian, ...) and belongs to the operator's kernel.

Joelle's option (1) of returning "the HR" is right in spirit; the precise refinement is to return the log Jacobian determinant.

The new contract

```java
public interface Scalable {
    double scale(double s); // returns log Jacobian
    double getScalableValue(); // current pos. on dilation axis
    default double setScalableValue(double V) {
        return scale(V / getScalableValue());
    }
}
```

Three methods, mutually consistent on a single dilation axis.

The three invariants

For any valid Scalable `x` and positive `s`:

1. **Scale-equivariance.** After `scale(s)`, `getScalableValue()` returns $s\times$ its previous value.

```
double v0 = x.getScalableValue();
x.scale(s);
assert x.getScalableValue() == s * v0;
```

2. **Set is a fixed point of get.**

```
x.setScalableValue(V);
assert x.getScalableValue() == V;
```

3. **Set composes with scale.**

```
x.setScalableValue(x.getScalableValue() * s) == x.scale(s);
```

The default `setScalableValue` expresses (3) directly.

What is the dilation axis?

`getScalableValue` must return whatever quantity *actually* scales by exactly `s` under the implementer's `scale`.

Implementer	Dilation summary
`RealScalarParam`	the value itself
`RealVectorParam`	sum of values
`Tree` (interval scaling)	sum of margins
`Tree` (affine height scaling)	root height
Joelle's custom tree	whatever her scale preserves

The contract is enforced per-implementation. A type that violates the invariants is not a valid Scalable.

Why interval scaling for `Tree.scale`?

The old `Tree.scale(s)` multiplied internal heights by `s` (affine). For a heterochronous tree with leaves at `h_c > 0` and parent at `h_N`, the move **throws** when `s < h_c / h_N`.

This is not an edge case. Most BEAST analyses with sampling dates run on heterochronous trees.

Interval scaling: multiply each internal node's margin (height above its taller child) by `s`.

- Tip dates preserved by construction.
- Move is valid for any positive `s`; never throws.
- Sum of margins is exactly `s`-equivariant: matches `getScalableValue`.
- For binary trees, the dof count equals the number of margins, so operator HRs are unchanged.

Affine semantics retained where needed

Some callers genuinely want "set my root to exactly this height" with affine semantics.

`Tree.scaleToRootHeight(double targetHeight)` preserves the old affine behaviour as a separate helper, outside the contract.

- Multiplies internal heights by `targetHeight / oldRoot`.
- May throw `IllegalArgumentException` for invalid configurations.
- Used by `StarBeastStartState` init code.
- Not part of the Scalable contract.

Two methods, two purposes: `scale` for moves, `scaleToRootHeight` for setup.

AMVN now reaches its proposed value

Before: AMVN proposed a target root height V, called `tree.scale(V/oldRoot)`.

For non-ultrametric trees under affine scaling, the move sometimes **threw**. For non-ultrametric trees under interval scaling, the root would end up at some height other than V. Either way, AMVN's empirical covariance bookkeeping was wrong.

After: AMVN parameterises trees by sum-of-margins via `getScalableValue` / `setScalableValue`. `setScalableValue(V)` reduces to `scale(V / sum_margins)`.

- Sum of margins equals V exactly, by the contract.
- AMVN's covariance is now updated against the realised state.
- Side benefit: fixes a latent missing `return` in spec AMVN's `getValue` for trees.

Test plan

Three layers of testing in PR #70:

- **ScalableContractTest harness**: a reusable helper that asserts the three invariants on any Scalable. Per-class tests for `RealScalarParam`, `RealVectorParam`, `Tree` on ultrametric, heterochronous, and leaf-intruding fixtures.
- **408 existing beast-base unit tests** all pass.
- **Heterochronous MCMC integration**: 10 `TipTimeTest` chains under `-Pslow-tests`. Real chains on tip-dated data converge to BEAST1 reference values within tolerance. *Tree height:* 15147 ± 73, *expected* 15000 ± 70, *ESS* $\sim$*22000.*

Statistical-tolerance fix on two slow tests (`UpDownOperatorTest`, `RealRandomWalkOperatorTest`): tolerances widened to 3 SE using ESS rather than nominal sample count, so they fail less often.

Downstream impact

Binding API change. Any package that implements `Scalable` or stores the int return needs to update.

Package	Impact	Status
Mascot	none (no `Scalable`)	compiles clean
beast-classic	none	compiles clean
LPhyBeast core	none	compiles clean
BEASTLabs	2 files	migrated, all 83 tests pass

BEASTLabs migration in parallel PR #29:

- `PrevalenceList.scale` returns log Jacobian; new `getScalableValue` returns sum of node times.
- `TreeScaleOperator` consumes log Jacobian directly; HR factored as `treeLogJ − 2 log(s)`.

Out of scope (deferred)

Operator-design questions, separable from the interface contract:

- Spec `UpDownOperator`'s `actualScaler = lengthAfter / lengthBefore` logic for tree + up/down combinations.
- Whether `IntervalScaleOperator` should collapse into `UpDownOperator` now that `Tree.scale` does interval scaling.
- AMVN's covariance behaviour for non-ultrametric trees beyond the proposal-target fix.
- Whether tree-aware operators should use the kernel `s` or an effective tree-dilation factor for non-tree up/down parameters.

The interface contract enables these to be tackled separately on their own merits.

Timeline

- **2 May**: PR #70 and BEASTLabs #29 ready for review.
- **Awaiting**: Remco's review (he owns beast-base).
- **Proposed at next stakeholders meeting**: this is the last substantive API change to the beast core before an official release. Aim for release on the BEAST website mid-June.

Summary

The contract

`scale(s)` returns the log Jacobian determinant of a move along a single dilation axis. `getScalableValue` reads position on that axis. `setScalableValue(V) = scale(V / getScalableValue())`.

The Tree side

`Tree.scale` does interval scaling and never throws. Sum of margins is the dilation summary. `Tree.scaleToRootHeight` preserves affine semantics where they were genuinely wanted.

The result

Joelle's custom trees can implement the contract directly. AMVN works correctly on non-ultrametric trees. $408 + 9 + 10$ tests pass. Mid-June beast3 release on track.